

Week 2 Deep Learning Technical Report

PyTorch Neural Networks, CNN Image Classification & Hyperparameter Experimentation

Name: Rakshitha M | Domain: AI & Machine Learning | Week: 2 | Date: 27 September 2026

1. Problem Statement & Dataset

This report summarizes three PyTorch deep learning tasks completed in Week 2: a feed-forward neural network for regression, a convolutional neural network (CNN) for image classification, and a set of controlled hyperparameter experiments. The goal was to build practical skill in the PyTorch training workflow — from defining models and loss functions to running full training loops and evaluating results — while learning how architectural and hyperparameter choices affect model performance.

Two datasets were used. The California Housing dataset (20,640 rows, 8 numeric features) was used for the regression task and the hyperparameter experiments, with the goal of predicting median house value for a district. The Fashion-MNIST dataset (70,000 28×28 grayscale images across 10 clothing categories) was used for the image classification task.

2. Architecture & Training Setup

Feed-forward network (Task 2.1): An 8→32→16→1 fully connected network with ReLU activations, trained with MSE loss and the Adam optimizer (lr=0.001) for 50 epochs on the California Housing dataset.

CNN (Task 2.2): Two convolutional layers (16 and 32 filters, 3×3 kernels) each followed by ReLU and max pooling, then two fully connected layers (1568→128→10). Trained with CrossEntropyLoss and Adam (lr=0.001) for 10 epochs on Fashion-MNIST.

Hyperparameter experiments (Task 2.3): Using the same feed-forward architecture as Task 2.1, 13 controlled experiments were run, changing one hyperparameter group at a time: learning rate, optimizer, batch size, and number of epochs.

3. Comparison Table & Key Findings

Task	Model	Key Setting	Train Metric	Test Metric
2.1	Feed-forward NN	Adam, lr=0.001, 50 epochs	Loss: 0.2745	Loss: 0.2921
2.2	CNN	Adam, lr=0.001, 10 epochs	Acc: 95.52%	Acc: 91.64%
2.3	Feed-forward NN (best run)	Adam, lr=0.001, batch=32, 30 epochs	Loss: 0.2780	Loss: 0.2915

Hyperparameter Varied	Best Setting	Worst Setting	Test Loss Range
Learning rate	0.001	0.0001	0.2915 - 0.3900
Optimizer	Adam	SGD	0.2915 - 0.4256
Batch size	32	128	0.2915 - 0.3167
Epochs	60	10	0.2807 - 0.3379

Across all experiments, learning rate had the single largest impact on performance — an overly small learning rate (0.0001) caused clear underfitting, while both 0.01 and 0.001 converged well. Optimizer choice mattered significantly at a fixed learning rate: SGD underperformed Adam and RMSprop by a wide margin (test loss 0.4256 vs ~0.29),

though this reflects an interaction with learning rate rather than SGD being categorically worse. Batch size showed a clear, consistent trade-off between accuracy and training speed — smaller batches converged to better loss values but took proportionally longer to train. Increasing epochs from 30 to 60 continued to improve test loss, but with diminishing returns relative to the added training time, and no overfitting was observed even at 60 epochs.

4. Generalization Assessment

All three tasks showed healthy generalization rather than overfitting. In Task 2.1, training and test loss remained close (0.2745 vs 0.2921). In Task 2.2, the CNN showed a modest, expected gap between training and test accuracy (95.52% vs 91.64%), consistent with normal generalization rather than memorization. In Task 2.3, even extending training to 60 epochs did not widen the gap between train and test loss, suggesting the model and dataset combination could tolerate additional training without overfitting risk.

5. Limitations & Next Steps

This week's work has some limitations worth noting. Each hyperparameter experiment was run only once with a fixed random seed, so small differences between close results (for example, learning rates 0.01 vs 0.001) may partly reflect run-to-run variance rather than a reliable difference. The CNN was trained for only 10 epochs; a longer training run or data augmentation could likely improve its 91.64% test accuracy further. Future work could include a systematic grid search across multiple hyperparameters simultaneously (rather than one group at a time), and repeating key experiments across multiple seeds to measure result stability.